

1. Introduccion

El presente informe tecnico tiene como objetivo documentar el proceso de auditoria realizado sobre un sistema bancario desarrollado en C++ bajo un enfoque Legacy, asi como el posterior rediseño de su estructura utilizando el lenguaje de programacion Java.

Durante la auditoria se analizaron principalmente los mecanismos de manejo de memoria mediante punteros, la creacion dinamica de estructuras y la liberacion de memoria. Se identifico que el sistema utiliza memoria dinamica para almacenar tanto la estructura CuentaLegacy como el nombre del titular, pero no realiza correctamente la liberacion de estos recursos al finalizar su utilizacion.

Posteriormente, se realizo un rediseño del sistema en Java utilizando programacion orientada a objetos. En este nuevo diseño se implemento una clase abstracta CuentaBancaria y las clases CuentaAhorros y CuentaCorriente, permitiendo representar los diferentes tipos de cuentas mediante herencia y polimorfismo.

Finalmente, se incorpore una clase RegistroAuditoriaBancaria que implementa AutoCloseable, utilizando la estructura try-with-resources para garantizar el cierre automatico del recurso utilizado para el registro de operaciones bancarias.

2. Objetivos

2.1 Objetivo general

Realizar una auditoria tecnica de un sistema bancario Legacy desarrollado en C++, identificar problemas relacionados con la gestion manual de memoria y rediseñar el sistema utilizando principios de programacion orientada a objetos en Java.

2.2 Objetivos especificos

- Identificar el uso de punteros y memoria dinamica en el codigo Legacy.
- Analizar los posibles problemas relacionados con la liberacion de memoria.
- Determinar el procedimiento correcto para liberar los recursos utilizados por el sistema.
- Rediseñar la estructura del sistema bancario utilizando clases en Java.
- Aplicar herencia y polimorfismo mediante una clase abstracta.
- Implementar diferentes comportamientos para las cuentas de ahorro y corriente.
- Incorporar un mecanismo de auditoria para registrar las operaciones bancarias.
- Utilizar AutoCloseable y try-with-resources para garantizar el cierre automatico del recurso de auditoria.

3. Descripción del sistema Legacy

El sistema original está desarrollado en C++ y utiliza una estructura denominada CuentaLegacy para representar una cuenta bancaria.

La estructura contiene los siguientes atributos:

- titular: puntero de tipo char* utilizado para almacenar dinámicamente el nombre del titular.
- saldo: almacena el saldo disponible de la cuenta.
- tipoCuenta: identifica si la cuenta es de ahorros o corriente.
- limiteSobregiro: representa el límite permitido para las cuentas corrientes.

La función crearCuenta() se encarga de crear dinámicamente una estructura CuentaLegacy mediante el operador new. Además, reserva un bloque adicional de memoria para almacenar el nombre del titular.

Por otro lado, la función procesarRetiro() recibe un puntero hacia una cuenta y modifica directamente su saldo dependiendo del tipo de cuenta y del límite de sobregiro disponible.

4. Auditoría del código C++

4.1 Uso de punteros

Durante la auditoría se identificaron dos elementos principales relacionados con punteros.

El primero corresponde al puntero CuentaLegacy* c. Este puntero almacena la dirección de memoria de una estructura CuentaLegacy creada dinámicamente mediante new CuentaLegacy().

El segundo corresponde al atributo char* titular. Este puntero almacena la dirección de un bloque de memoria reservado dinámicamente mediante new char[50].

Por lo tanto, existen dos reservas de memoria independientes que deben ser gestionadas correctamente.

4.2 Problema de gestión de memoria

El principal problema identificado durante la auditoría es que el código no contiene instrucciones para liberar la memoria reservada dinámicamente.

La estructura es creada mediante CuentaLegacy* c = new CuentaLegacy(); mientras que el nombre del titular es reservado mediante c->titular = new char[50].

Sin embargo, no se encuentran las instrucciones correspondientes para liberar estos recursos.

Esto puede provocar fugas de memoria, debido a que los bloques reservados permanecen ocupados aunque ya no sean necesarios.

4.3 Orden correcto para liberar la memoria

El orden de liberación es importante debido a que titular pertenece a la estructura CuentaLegacy.

Primero debe liberarse la memoria apuntada por titular mediante `delete[] c->titular`.

Posteriormente debe eliminarse la estructura mediante `delete c`.

El procedimiento correcto es, por lo tanto:

```
delete[] c->titular;  
delete c;
```

Primero se libera el arreglo dinámico utilizado para almacenar el titular y después se libera la estructura que contiene el puntero.

Si se eliminara primero la estructura mediante `delete c`, se perdería el acceso al puntero titular contenido dentro de ella, provocando que la memoria reservada para el titular no pueda ser liberada correctamente.

5. Problemas identificados en la auditoría

A partir del análisis realizado se identificaron los siguientes problemas:

5.1 Gestión manual de memoria

El código utiliza `new` para realizar reservas dinámicas de memoria, lo que obliga al programador a liberar posteriormente dichos recursos.

5.2 Fugas de memoria

No existe un procedimiento de liberación de la memoria utilizada por titular ni de la estructura CuentaLegacy.

5.3 Uso de punteros

El sistema depende directamente de punteros para administrar las cuentas y los datos de los titulares, aumentando la posibilidad de errores relacionados con la memoria.

5.4 Falta de encapsulamiento

La estructura CuentaLegacy permite acceder directamente a sus atributos. No existe una separación clara entre los datos y las operaciones que pueden realizarse sobre ellos.

5.5 Falta de abstracción

Los diferentes tipos de cuenta son manejados mediante valores numéricos, donde 1 representa una cuenta de ahorros y 2 una cuenta corriente. Esto limita la extensibilidad del sistema.

5.6 Falta de auditoría de operaciones

El código original no dispone de un mecanismo formal para registrar las operaciones realizadas sobre las cuentas.

6. Rediseño del sistema en Java

Como solución a las limitaciones encontradas en el código Legacy, se realizó un rediseño utilizando Java y el paradigma de programación orientada a objetos.

La estructura principal está representada mediante la clase abstracta CuentaBancaria.

Esta clase contiene los atributos comunes de todas las cuentas:

- titular
- saldo
- tipoCuenta
- limiteSobregiro

Los atributos se definieron como privados, permitiendo aplicar encapsulamiento mediante métodos get y set.

7. Clase abstracta CuentaBancaria

La clase CuentaBancaria funciona como una abstracción para representar las características comunes de los diferentes tipos de cuentas bancarias.

Además, define dos métodos abstractos:

```
procesarRetiro(double monto)
aplicarComisionMensual()
```

Estos métodos obligan a las clases derivadas a implementar su propio comportamiento.

De esta manera, la clase principal establece que operaciones debe soportar una cuenta, mientras que cada tipo de cuenta determina cómo deben ejecutarse.

8. Implementación de AutoCloseable

Como parte de la tercera fase del proyecto se creó la clase RegistroAuditoriaBancaria.

Esta clase implementa la interfaz AutoCloseable, permitiendo que pueda ser utilizada dentro de una estructura try-with-resources.

El objetivo es simular un recurso destinado al registro de operaciones bancarias y garantizar que sea cerrado automáticamente cuando termine su utilización.

La clase contiene un método registrar() para mostrar los eventos de auditoría y un método close() encargado de cerrar el recurso.

9. Uso de try-with-resources

El recurso de auditoría se utiliza mediante:

```
try (RegistroAuditoriaBancaria auditoria = new RegistroAuditoriaBancaria())
```

Dentro del bloque try se registran diferentes operaciones bancarias.

Una vez finalizado el bloque, Java ejecuta automaticamente el metodo close() del objeto.

Este mecanismo garantiza que el recurso sea cerrado sin que el programador tenga que llamar manualmente al metodo close().

Ademas, el cierre del recurso se realiza incluso cuando se presenta una excepcion durante la ejecucion del bloque, lo que proporciona un mecanismo mas seguro para la gestion de recursos.

10. Ventajas del rediseño

El rediseño realizado en Java presenta varias ventajas frente al sistema original.

Se reduce la complejidad relacionada con la gestion manual de memoria, ya que Java utiliza un recolector de basura para administrar los objetos que dejan de ser utilizados.

Tambien se incorpora herencia mediante CuentaAhorros y CuentaCorriente, permitiendo reutilizar los atributos y comportamientos comunes definidos en CuentaBancaria.

El polimorfismo permite que cada tipo de cuenta implemente de manera diferente operaciones como los retiros y las comisiones mensuales.

Finalmente, AutoCloseable y try-with-resources permiten realizar una gestion mas segura y estructurada del recurso utilizado para la auditoria bancaria.

11. Conclusiones

La auditoria del codigo Legacy permitio identificar problemas relacionados principalmente con la gestion manual de memoria. El sistema utiliza reservas dinamicas mediante new tanto para la estructura CuentaLegacy como para el almacenamiento del nombre del titular, pero no realiza la liberacion correspondiente.

Se determino que para liberar correctamente la memoria es necesario eliminar primero el arreglo dinamico apuntado por titular mediante delete[] y posteriormente eliminar la estructura CuentaLegacy mediante delete. Este orden evita que se pierda el acceso al bloque de memoria reservado para el titular.

El rediseño en Java permitio mejorar la organizacion y mantenibilidad del sistema mediante programacion orientada a objetos. La utilizacion de una clase abstracta, herencia, encapsulamiento y polimorfismo permite representar de una manera mas clara los diferentes tipos de cuentas bancarias.

En conclusion, el proceso de migracion del sistema Legacy hacia Java permite obtener una estructura mas organizada, mantenible y segura, reduciendo los riesgos asociados a la gestion manual de memoria y proporcionando mecanismos adecuados para la auditoria de las operaciones bancarias.